教育部直属国家211工程重点大学
华中农业大学
HUAZHONG AGRICULTURAL UNIVERSITY

大学数学基础实验

——贪婪算法浅析

--大学数学实践教学系列课件--


中国人民银行
伍分币壹千枚伍拾元整
伍分
1989

内容提要

1 贪婪法

2 背包问题

贪婪法

找硬币：

假设有四种硬币，它们的面值分别为2角5分、1角（1角=10分）、5分和1分。如何找给某顾客六角三分钱？

2个25分+1个10分+3个1分的硬币

63　-25　-25　-10　-1　-1 -1 =0

这种找硬币方法与其他的找法相比，所拿出的硬币个数是最少的。

- 顾名思义，贪婪算法总是以当前情况为基础，而不考虑全部各种可能的情况，只作出在当前状态看来是最好的选择。也就是说贪婪算法并不从整体最优上加以考虑，它所作出的选择只是在某种意义上的局部最优选择。
- 当然，我们希望贪婪算法得到的最终结果也是整体最优的。上面所说的找硬币算法得到的结果就是一个整体最优解。

❖贪婪法找硬币的matlab实现

```
>>coins=[25,10,5,1];total=63;%初始值
rest=total;i=1;change=[];
coins=sort(coins, 'descend')%降序排列
while rest>0
        if (rest-coins(i))>=0
                change =[change,coins(i)];
                 rest=rest-coins(i);
        else
                 i=i+1;
        end
end
change%找的零钱一览
length(change)%零钱个数
```

- 再看找硬币：
- 如果变为考虑1、8、9、11这四种面值的硬币，要找出币值24的零钱，怎么找能使硬币数最少？

```
change =

    11    11     1     1


ans =

     4
```

- 在一些情况下，贪婪算法不一定能找到最优解。

- 即使贪婪算法不能得到整体最优解，然而，却是一种具有直觉的倾向且一般情况下其最终结果仍是最优解的很好的近似解。
- 它利用的规则就是在实际环境中人工所采用的规则。作出贪婪决策的依据称为贪婪准则（**greedy criterion**）
- 最朴素的启发式算法
- 优点：简单易行，省去了为找最优解要穷尽所有可能而必须耗费的大量时间，构造贪心策略不是很困难，通常得到的结果可以是最优解的一个近似解。

背包问题

设有n=8个体积分别为1 10 20 40 45 22 30 55，价值分别为10 20 30 50 55 32 40 60的物品，和一个容积为C=110的背包，问选择哪几个物品装入背包可以使其装的最满。

直接用贪婪算法：仅仅只考虑体积。将物体由大到小顺次装入背包，到装不下时再逐个试装更小的一些，直至试到最小的一个或装满为止。按此处所给数据，先装入55和45两个，容积尚余10，最后再装入体积为10的一个，总体积达到110，装满。此方法的好处是快速。

```
function [sch,tolwei,tolval]=backpack1(maxwei,weight,value)
n=size(weight,2);sch=zeros(1,n);
p=weight;
[a,b]=sort(p,'descend');%a从大到小排序后的向量,b是对应元素原始下标
tw=0;%已装入背包的物品体积
for i=1:n
    if (tw+weight(b(i)))<=maxwei
        tw=tw+weight(b(i));
        sch(b(i))=1;
    end
end
tolwei=tw;tolval=sum(value(find(sch)));
```

```
>> [s,t,v]=backpack1(110,[1 10 20 40 45 22 30 55],[10 20 30
50 55 32 40 60])
s =
    0    1    0    0    1    0    0    1
t =   110                      t =    132
```

本问题的做法2：

先计算物品的价值密度，并把物品按价值密度从大到小的顺序排列：

$$p_i = \frac{v_i}{w_i}, i = 1,2,...,n \quad p_{k_1} \geq p_{k_2} \geq ... \geq p_{k_n}$$

当选择第k_i件物品时，先判断背包是否超载，如果不超载，则放入背包，否则考虑下一件k_{i+1}。按照这种方式考虑所有物品，即能得到背包问题的一个近似最优解。

```
%%
p=value./weight;%将背包函数文件第三行代码改为
%%
```

```
>> [s,t,v]=backpack1(110,[1 10 20 40 45 22 30 55],[10 20 30
50 55 32 40 60])
s =
     1     1     1     0     0     1     1     0
t= 83              v =   132
```